MEGATHON MID 1 SUBMISSION

Team “team”

1. Problem Statement and Understanding

Wikimedia Commons contains millions of images organized into a large hierarchy of categories. Manually assigning appropriate categories to every image is time-consuming and requires knowledge of the subject, context, and existing category structure. The objective of MemoryTag AI is to automate this process by analyzing an uploaded image and suggesting relevant categories.

The challenge extends beyond recognizing everyday objects. Images may contain multiple objects, depict scenes or events, or represent abstract concepts. Furthermore, useful suggestions should correspond to existing Wikimedia Commons categories rather than merely producing generic labels. Therefore, the system must combine visual understanding, multi-label classification, category retrieval, and relevance-based ranking.

2. Idea or Abstract

MemoryTag AI is an image-categorization system designed to generate accurate, relevant category suggestions for Wikimedia Commons. Our initial implementation establishes a multi-label image classifier using transfer learning with a pretrained ResNet18 model. Rather than training a convolutional neural network entirely from scratch, we reuse visual features learned from a large image dataset and adapt the model to recognize ten COCO categories: person, bicycle, car, motorcycle, airplane, bus, train, truck, boat, and traffic light.

This implementation serves as a working baseline. The planned system will extend beyond these fixed categories using a pretrained vision-language model to understand unfamiliar images, generate descriptive labels, and retrieve suitable existing Commons categories. A ranking component will prioritize suggestions according to visual relevance, textual similarity, and category relationships.

3. High-Level Approach and Architecture

Our development process follows these stages:

Dataset preparation: We downloaded a 10,000-image subset of the COCO training dataset. It contains 8,000 images with annotations for at least one of the ten target categories and 2,000 images without annotations for those categories. Images without target annotations were assigned all-zero target vectors, allowing the model to learn when none of the selected categories should be predicted. However, such images may still contain unannotated target objects.

We created disjoint training, validation, and test sets using : 8,000 training images, 1,000 validation images, and 1,000 test images.

Transfer learning: We loaded pretrained ResNet18 weights and replaced the original classification head with a fully connected layer containing ten outputs. Each output represents one category. Sigmoid activations convert logits into category scores, while binary cross-entropy with logits supports independent multi-label predictions.

Because categories occur at different frequencies, we used weighted loss through pos_weight, calculated from negative-to-positive label ratios and capped at ten. This gives less frequent categories greater influence during training.

Training and fine-tuning: Initially, we froze the pretrained feature extractor and trained only the new classification head for five epochs. We monitored validation loss and saved the best checkpoint. We then unfroze the final convolutional block (layer4) and classification head, using smaller learning rates to fine-tune the pretrained features.

Across four additional epochs, validation loss improved from 0.5495 for the best initial model to 0.5288 for the best fine-tuned checkpoint. We retained the best-performing checkpoint rather than automatically choosing the final epoch. This checkpoint-based selection, together with validation monitoring, helped prevent retaining a model whose performance had deteriorated in later epochs. We did not repeatedly duplicate validation cases. instead, we reused the same validation set to compare model checkpoints consistently.

Threshold optimization and inference: Initially, every category used a threshold of 0.50. We then selected a separate threshold for each category by maximizing its F1 score on the validation set. The overall validation micro F1 increased from 0.677 to 0.742. These thresholds were saved and incorporated into a reusable predict_image() function, which returns ranked category scores and selected tags.

Planned extension: We intend to extend to more categories and subcategories. We also intend to add an interactive UI using flask to assist every user.

4. Planned Testing Methodology

We use three evaluation stages. During training, validation loss guides checkpoint selection. On the validation set, we measure per-category precision, recall, F1 score, and average precision (AP), then tune classification thresholds. Finally, we evaluate the selected checkpoint and fixed validation-derived thresholds on the untouched test set.

Our test results were precision 0.691 and recall 0.756. We also tested individual positive and negative examples to verify the inference pipeline. Future evaluation will measure category relevance, ranking quality, coverage of diverse image concepts, and the validity of suggested Commons categories.